# Assignment 2: Simple Search Engine using Hadoop MapReduce

Salavat Faizullin DS-01

The 1-st of April, 2026

## 1 Methodology

My goal was to build a distributed search engine that can index text documents and rank them using the BM25 algorithm. To do this, I split the system into four main parts: data preparation, MapReduce indexing, database storage, and Spark ranking.

### 1.1 Data Preparation

First, the pipeline demands the dataset to be in the HDFS structure. I wrote a preparation script that copies the `a.parquet` file into the container's HDFS. To build the initial corpus, PySpark DataFrame API was used strictly reading from HDFS to extract a sample of 1000 plain text documents, which were saved locally and re-uploaded as text files. Finally, following the assignment rules closely, using PySpark's `sc.wholeTextFiles()`, I loaded all the generated `.txt` files in a distributed resilient dataset, transformed the RDD, and saved it into HDFS under the folder `/input/data` as one single partition with tab separations.

### 1.2 Two-Stage Hadoop MapReduce Indexer

BM25 scoring needs both global numbers-like how many documents there are, how many times a term appears across all documents, and the average document length-and local numbers like how many times a term appears in a specific document and the length of that document. To get all these efficiently, I made a two-pipeline MapReduce system using Python and Hadoop Streaming.

In the first pipeline, the mapper goes through each document, cleans up the text(lowercase, removes punctuation), and for every word it sends out a key that combines the word and the document ID with a value of 1. It also counts the total words in the document and sends out a special marker. The reducer then adds up the counts, giving the exact term frequency for each word inside each document, and passes the markers along.

In the second pipeline, another mapper reads the output from the first step. It separates the regular term records from the document length markers. The reducer then counts how many documents each term appears in(that's the document frequency) and also adds up all the document lengths to find the average. Finally, it writes out three kinds of records: the index entries, the document metadata, and the global statistics.

### 1.3 Database Storage with Cassandra/ScyllaDB

To make queries fast, I needed to move the results from HDFS into a real database. I have finished a script called store_index.py that uses the cassandra-driver package. I set up four tables inside the search_engine keyspace. One table stores the global statistics, another holds each document's title and length, a third keeps the term frequencies for each term and document, and the fourth lists each term with its global document frequency. This design makes it easy to look up information when answering a query.

### 1.4 Spark RDD Ranker

The final piece is the query script, written strictly with PySpark's RDD API. When the user executes a query, the script accepts it via `stdin` or command line arguments. Then it goes to the Cassandra nodes to retrieve the inverted index entries for the query terms and fetches the global metrics like total documents $N$ and average document length $dl_{avg}$.

To ensure the BM25 logic executes in a fully distributed way instead of looping on the single cluster driver, the relevant Document IDs mapped with their term frequencies are parallelized. I used Spark RDD methods to aggregate the records through `groupByKey()`, grouping all matching query terms per document. Each worker calculates the accumulated BM25 score for the documents assigned to it using the following algorithm:

$$BM25(q,d) = \sum_{t \in q} IDF(t) \cdot \frac{tf(t,d) \cdot (k_1 + 1)}{tf(t,d) + k_1 \cdot \left(1 - b + b \cdot \frac{dl(d)}{dl_{\text{avg}}}\right)}$$

I set the parameters to the usual defaults, $k_1 = 1.0$ and $b = 0.75$. After all distributed workers finish the math, the script uses `sortBy()` efficiently returning the Top 10 descending BM25 scores alongside their IDs.

### 1.5 Cluster Execution and Packaging

One tricky part was that the YARN containers did not have the Python libraries I needed, such as the Cassandra driver. To solve this, I used venv-pack to bundle all dependencies into a virtual environment. The app.sh script creates this environment, packs it into a .tar.gz file, and sends it to the Spark workers using the –archives flag. This way, every worker had the necessary packages available.

## 2 Demonstration

### 2.1 Starting the Infrastructure

Running everything is fully automated with Docker Compose. However, to execute the pipeline properly, you must insert the `a.parquet` dataset directly into the `app/` directory before initiating. Because the dataset is practically 6 GB, it was logically excluded from the git repository.

After dropping the file, open a terminal in the root directory and run `docker compose up -d`. That instantiates the master, slave, and Cassandra server containers. It automatically executes the `app.sh` orchestrator inside the master node. After waiting for PySpark Dataframe slicing, HDFS staging, mapping and reducing, and NoSQL insertions, the prompt will begin executing the demo string queries dynamically.

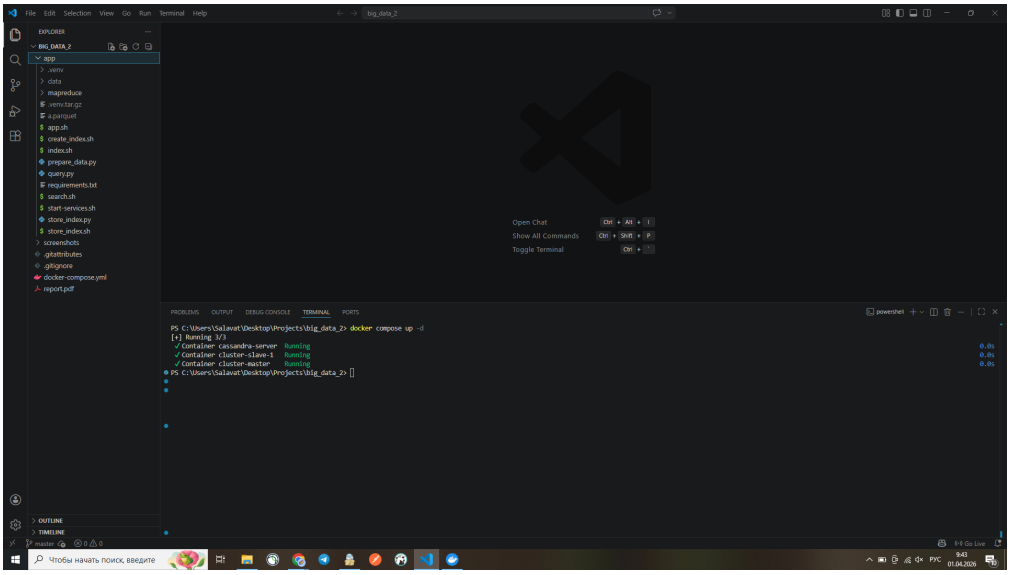


Figure 1: Screenshot 1: Starting the infrastructure(Hadoop, Spark, Cassandra) via terminal.

## 2.2 Indexing Execution

The screenshot below shows how the documents were transformed, uploaded to HDFS, and processed through the two MapReduce steps. You can see the logs from the YARN nodes showing the map and reduce tasks completing.

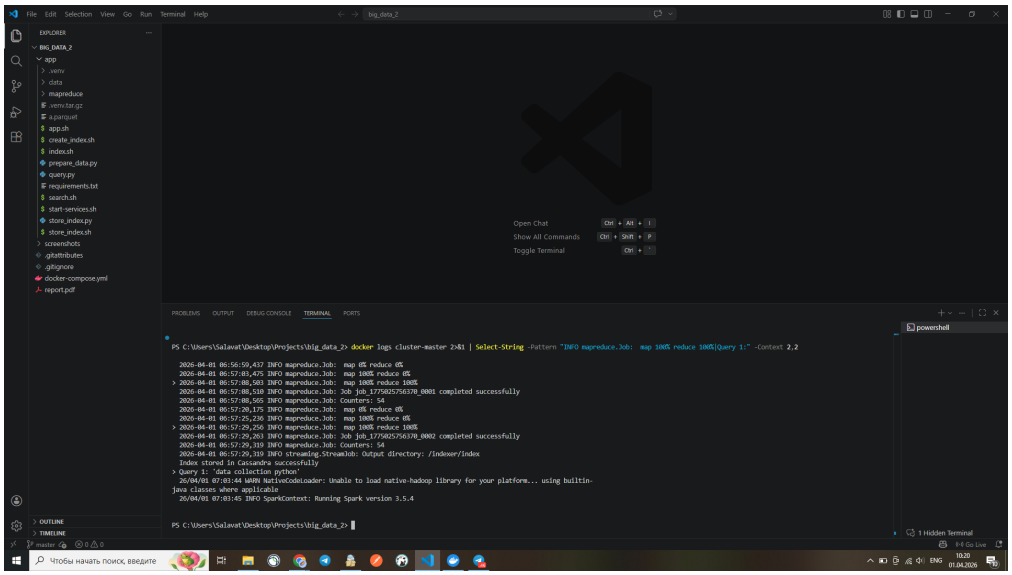


Figure 2: Screenshot 2: Successful completion of MapReduce indexing and saving dictionaries to Cassandra.

## 2.3 Query Answering Results

I tested two queries to see how well the search engine worked.

The first query was “data collection python”. The system looked up the terms, fetched the necessary data from Cassandra, and used Spark to calculate the BM25 scores. The top results came out as shown.

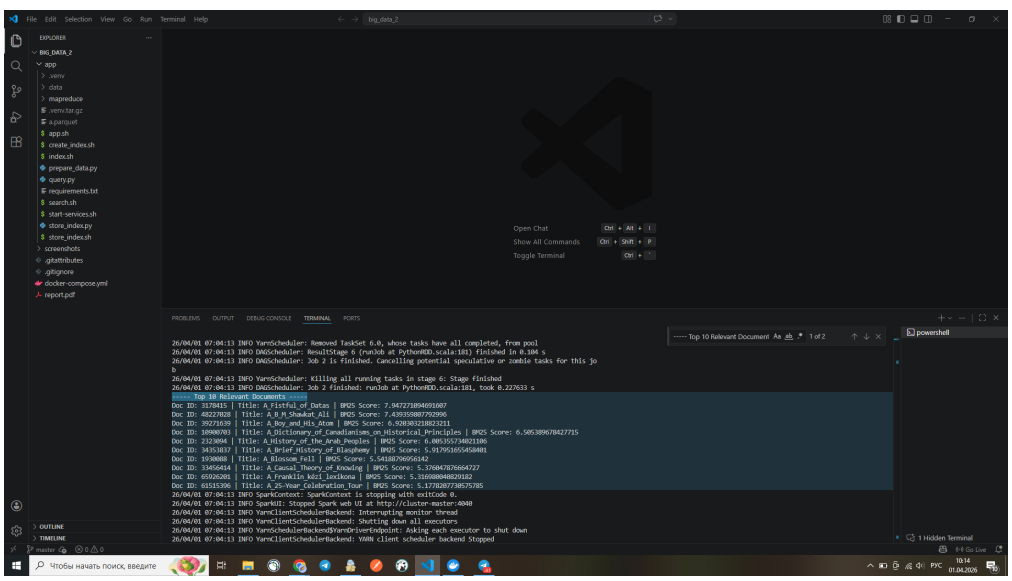


Figure 3: Screenshot 3: Execution of Query 1 ("data collection python") and output of the Top-10 relevant documents.

The second query was “distributed systems mapreduce”. The search returned documents

that matched well, including files like A_Fistful_of_Datas, which had high relevance scores. The whole operation finished cleanly with a success code.

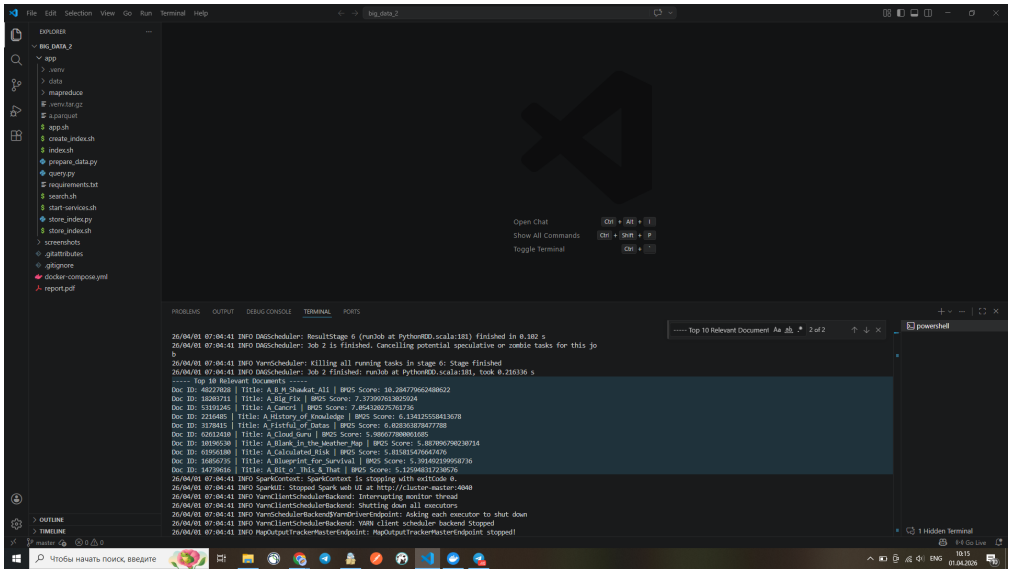


Figure 4: Screenshot 4: Execution of Query 2 ("distributed systems mapreduce") and output of the Top-10 relevant documents.

## 2.4 Conclusion and Findings

By manually implementing TF/IDF and BM25, I got very relevant search results. Storing the vocabulary in a partitioned Cassandra table made lookups fast, much better than scanning HDFS directories every time. Splitting the indexing into two MapReduce stages kept the system from running out of memory. I also made sure not to use any single-node libraries like pandas-everything was designed to run on a cluster, just as the assignment required.